

Python Backend Development Notes

UV, Docker, FastAPI Security, JWT, Architecture, Migrations & Testing

Simple + detailed summary of this conversation

Core idea

A modern FastAPI project can use UV to manage Python dependencies, Docker to package and run the app consistently, JWT + bcrypt for authentication, Route/Service/Repository layers for clean architecture, Alembic for database schema history, and isolated Postgres/Redis environments for safe testing.

1. Big Picture

The main concepts we discussed fit together like this:

Developer machine

- |
- +-- uv / virtual environment
- +-- pyproject.toml -> what dependencies the project needs
- +-- uv.lock -> exact dependency versions
- |
- +-- Dockerfile
 - > Docker build
 - > image
 - > container running FastAPI

Inside the application:

Request -> security checks -> Route -> Service -> Repository -> PostgreSQL

- |
- +--> Redis for token state / revocation

Database structure -> Alembic migrations

Initial/demo data -> seed.py

Tests -> real Postgres + Redis, but isolated test resources

2. UV and the uv.lock File

UV is a fast Python package and project manager. In a typical UV project, pyproject.toml and uv.lock have different jobs.

File	Simple purpose
pyproject.toml	Says which packages the project needs, plus project configuration.
uv.lock	Records the exact versions UV resolved for the

	project, including dependency versions pulled in by other packages.
Easy way to remember pyproject.toml = what I want. uv.lock = the exact package versions that were resolved.	

Why uv.lock matters: two developers or a developer and a CI/Docker build can install the same dependency versions instead of silently getting newer versions later.

uv sync

When you run uv sync, UV synchronizes the virtual environment from the project dependency definition and lock information.

3. How uv.lock Works With Docker

Docker does NOT automatically use uv.lock just because the file exists. Docker follows the instructions in the Dockerfile.

```
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen
```

COPY makes the lock file available inside the image build. uv sync --frozen tells UV to use the existing lock file exactly and not re-resolve dependencies.

Important distinction docker build = follow the Dockerfile. The Dockerfile decides whether and how uv.lock is used.
--

4. Docker: Image vs Container and Where FastAPI Goes

We discussed a common confusion: whether your FastAPI app is automatically "in Docker". The answer is: only when the image build puts your application files into the image and a container is created from that image.

```
Your source code
-> Dockerfile instructions
-> docker build
-> Docker image
-> docker run / compose
-> Docker container
-> FastAPI process running inside the container
```

Concept	Simple meaning
Dockerfile	Instructions for building the image.
Image	Packaged template containing your app, runtime, dependencies, and files.
Container	A running instance of an image.

Example

If the Dockerfile copies your FastAPI source into the image and installs its dependencies, then a container created from that image can run the FastAPI app inside Docker.

5. Running FastAPI With UV and Hot Reload

Hot reload means the development server watches your source files and restarts the application when you change code. It is a developer convenience; it is not a production requirement.

```
uv run uvicorn app.main:app --reload
```

Part	Meaning
uv run	Run the command using the UV-managed project environment.
uvicorn	ASGI server used to run the FastAPI application.
app.main:app	Import app object from app/main.py.
--reload	Watch for code changes and restart automatically during development.

6. Security Code: What It Is Doing

The security code has two main responsibilities: protecting passwords with bcrypt and creating/verifying JWT-based tokens.

6.1 Password Hashing With bcrypt

```
hash_password(password)
-> bcrypt.hashpw(...)
-> stored password hash
```

A password is not stored directly. bcrypt turns it into a one-way password hash. A salt is added so identical passwords do not all produce the same stored value.

The code uses bcrypt cost/rounds = 12. The comment says this is an intentional trade-off between security and server CPU time.

6.2 Verifying a Password

```
verify_password(password, password_hash)
-> bcrypt.checkpw(...)
-> True or False
```

At login, the user enters a raw password. bcrypt checks it against the stored hash. The application does not need to decrypt anything because bcrypt hashes are not meant to be decrypted.

6.3 JWT and Access Token

Simple difference

JWT is a token format. Access token is a token used to access protected APIs. In this project, the access token is a JWT.

The same idea applies to the refresh token: it can also be a JWT, but its purpose is different.

6.4 What the JWT Payload Contains

sub = user ID
jti = unique token ID
type = access or refresh
team_id = team ID
role = user role
iat = issued-at time
exp = expiration time

The token is signed using a secret and an algorithm from application settings. The signature lets the server detect a token that was modified or was not created by the server.

6.5 Access Token vs Refresh Token

Token	Purpose	Typical lifetime in this project
Access JWT	Used for protected API requests.	15 minutes
Refresh JWT	Used to obtain a new access token without making the user log in again.	7 days

6.6 How Refresh Tokens Create New Access Tokens

The refresh token itself does not magically create another token. The backend receives the refresh token, validates it, identifies the user/session, and then calls the token creation code to issue a new pair.

Login
-> Access token + Refresh token

Access token expires
-> Client sends refresh token
-> Backend validates refresh token
-> Backend issues new access token
-> Often also rotates the refresh token

Client
-> uses new access token for future API calls

7. The Request Pipeline

Bearer eyJ...
-> decode JWT
-> type == access?
-> blacklisted?
-> user exists and active?
-> require_role(...)
-> Route
-> Service
-> Repository
-> PostgreSQL

- > commit
- > audit row
- > ApiResponse[T]

This is a security and application-flow checkpoint. The API should reject bad or unauthorized requests before business logic performs the protected operation.

8. Route -> Service -> Repository -> Database

Layer	Purpose	Typical job
Route	Handle HTTP request/response.	Read input, invoke dependencies/service, return response.
Service	Business logic.	Apply rules such as permissions, validation, workflows, and decisions.
Repository	Database communication.	Queries and persistence operations such as get/create/update/delete.
Database	Store persistent data.	PostgreSQL stores users, teams, monitors, audit logs, etc.
Easy analogy Route = waiter taking the order. Service = chef deciding how the order should be prepared. Repository = kitchen staff retrieving/saving ingredients/data. Database = storage room where data lives.		

9. RBAC - Role-Based Access Control

RBAC means the permissions a user gets depend on their role.

Role	Example permissions
viewer	Read-only access.
member	Read plus create/edit monitors.
owner	Broader permissions such as invites, deletion, billing, and audit access.

```
require_role("owner")
```

In the discussed FastAPI design, `require_role` is a dependency factory. FastAPI runs the dependency before the route handler and can inject the authenticated User into the route when the check succeeds.

10. Redis, Refresh Tokens, Rotation, and Logout

The project tracks refresh-token state in Redis instead of treating a refresh JWT as the complete source of truth.

`refresh:{user}:{jti} = 1` (TTL about 7 days)

The refresh token contains a JTI (JWT ID). Redis stores whether that specific refresh JTI is still valid.

10.1 Refresh Token Rotation

Use refresh token

- > read JTI
- > check Redis
- > if valid: delete old JTI
- > issue a new access + refresh pair
- > store new refresh JTI

Why this helps: a refresh token is intended to be used once per rotation. If an old token shows up again after rotation, the server can treat it as a possible theft/reuse event.

10.2 Reuse/Theft Detection

Old refresh JTI no longer exists in Redis

- > possible stolen/reused refresh token
- > invalidate the refresh-token family
- > audit event
- > return 401

10.3 Logout and Blacklisting Access JWTs

`jwt:black:{jti} = 1`

TTL = remaining lifetime of the token

JWTs are commonly stateless: once issued, the server normally does not keep a list of every access token. Because of that, the server cannot simply "unsign" a JWT during logout. A blacklist/revocation record in Redis is a practical way to reject that token before it expires naturally.

11. Registration and Login Flow

11.1 Registration

POST /auth/register

- > Pydantic validates input
- > validate password length / bytes
- > bcrypt hash
- > begin DB transaction
- > create Team
- > create User(role=owner)
- > create audit row
- > commit

The transaction is described as all-or-nothing: if something fails before commit, the intended result is that the related database changes do not become partially saved.

Bcrypt 72-byte detail

bcrypt historically limits the password input to 72 bytes. The project intentionally rejects passwords beyond this limit rather than silently truncating them.

11.2 Login

POST /auth/login

- > select user by email
- > bcrypt check
- > success: issue access + refresh tokens
- > store refresh JTI in Redis
- > audit login

Failure:

- > audit login_failed
- > same 401-style failure
- > avoid revealing whether the email exists

Returning the same general failure for wrong email and wrong password helps reduce user-enumeration attacks.

12. Database Concepts We Discussed

Concept	Simple meaning
UUID primary key	Uses a globally unique ID instead of a simple incremental integer. Useful for distributed systems and safer-looking public IDs.
ON DELETE CASCADE	When a parent row is deleted, related child rows are automatically deleted. Example: delete team -> team users are removed.
ON DELETE SET NULL	When the referenced row is deleted, the child keeps existing but the foreign-key value becomes NULL. Useful for preserving audit history.
Explicit constraint names	Database constraints get stable names, making migration changes/removals easier.
Enum stored as VARCHAR	Roles are stored as text values such as owner/member/viewer rather than a database enum type.
Alembic	Tool that tracks and applies database schema changes over time.
Easy comparison Migration = change the database structure. Seed = insert initial/demo data.	

13. seed.py - Demo/Initial Data

The seed script creates a demo team and owner account so the application has usable data immediately after the database schema exists.

Run:

```
uv run python scripts/seed.py
```

Prerequisite:
alembic upgrade head

The script first checks whether the demo email already exists. If it exists, it skips the seed so repeated runs do not create the same demo account again.

1. SELECT user by DEMO_EMAIL
2. If found -> print "Seed skipped" and return
3. Create Team(plan=FREE)
4. Flush so team.id is available
5. Create User(role=OWNER)
6. Hash the demo password with bcrypt
7. Commit
8. Print success message

Remember

seed.py creates data inside existing tables. It does not create the tables themselves.

14. Alembic migrations/env.py

Alembic uses env.py as configuration/glue code. It tells Alembic where the database is and which SQLAlchemy model metadata should be compared when autogenerating migrations.

```
config.set_main_option("sqlalchemy.url", get_settings().database_url)
```

This means the database connection comes from application settings/environment rather than being hardcoded into alembic.ini.

```
target_metadata = Base.metadata
```

This tells Alembic what the expected schema looks like from the SQLAlchemy model definitions.

```
alembic upgrade head
```

This applies all migrations up to the latest migration, often called head.

Best mental model

Alembic is like Git for DATABASE STRUCTURE, not Git for application data.

15. Test Architecture

The project intentionally tests against real Postgres and Redis instances launched by Docker Compose, but keeps test resources separated from development resources.

Development
PostgreSQL -> development DB
Redis -> Redis DB 0

Testing
PostgreSQL -> pulsetrack_test
Redis -> Redis DB 1

Before a test (as described), the test database schema is dropped and recreated. This produces a clean environment rather than reusing old test state.

FastAPI `dependency_overrides` are used so the application points its dependencies at the test database/Redis instead of the normal development resources.

Why this architecture is useful

You get realistic integration behavior because the tests talk to real services, while still protecting development data by isolating the test resources.

16. One End-to-End Mental Model

SIGN UP

- > validate request
- > bcrypt password
- > transaction creates team + user
- > commit

LOGIN

- > find user
- > bcrypt verification
- > access JWT + refresh JWT
- > store refresh JTI in Redis

API CALL

- > Bearer access JWT
- > decode/verify JWT
- > check token type
- > check blacklist
- > load/check active user
- > check role
- > Route
- > Service
- > Repository
- > PostgreSQL

ACCESS TOKEN EXPIRES

- > send refresh token
- > validate refresh JTI in Redis
- > rotate tokens
- > new access token

LOGOUT

- > blacklist access JTI in Redis
- > future requests with that token are rejected

SCHEMA CHANGE

- > change SQLAlchemy model
- > create Alembic migration
- > alembic upgrade head

DEMO DATA

- > run seed.py
- > insert demo team + owner

17. Quick Cheat Sheet

Term	One-line meaning
UV	Python project/dependency manager.
pyproject.toml	Project dependency/config declaration.
uv.lock	Exact resolved dependency versions.
Dockerfile	Build instructions for a Docker image.
Image	Packaged app environment/template.
Container	Running instance of an image.
FastAPI	Web API framework used by the application.
Uvicorn	ASGI server that runs the FastAPI app.
bcrypt	Password hashing algorithm/library.
JWT	Signed token format carrying claims.
Access token	Token used to call protected APIs.
Refresh token	Longer-lived token used to obtain new access tokens.
Redis	Fast in-memory store; here used for refresh-token state and revocation/blacklisting.
RBAC	Permissions based on user role.
Route	HTTP/API entry point.
Service	Business logic layer.
Repository	Database access layer.
Alembic migration	Versioned database schema change.
seed.py	Creates initial/demo database data.